Adobe Experience Manager

Study Guide for Beginners

Visual companion to the AEM introduction presentation

A complete walkthrough of architecture, core concepts, and how AEM is used in real-world scenarios.

Table of contents

1. Introduction 3
2. What is AEM? 3
The four main pillars 3
3. AEM architecture 4
Basic request flow 4
4. JCR (Content Repository) 5
Key paths 5
5. Apache Sling 6
Anatomy of a Sling URL 6
6. OSGi 7
Why OSGi matters 7
7. Components 8
Anatomy of a component 8
HTL: HTML Template Language 8
8. Editable templates 9
Two main areas 9
Policies define the rules 9
9. Content Fragments vs Experience Fragments 10
Content Fragment 10
Experience Fragment 10
10. Sling Models 11
Key annotations 11
11. AEM as a Cloud Service 12
What changes 12
Important rules 12
12. Author vs Publish 13
Author 13
Publish 13
The Dispatcher 13
13. Closing 14

1. Introduction

This guide walks through Adobe Experience Manager (AEM): what it is, how it works internally, and the key concepts you need to start working with it.

We move from high-level ideas down to the core building blocks, in a practical and easy-to-follow way. The goal is that, by the end, you can confidently navigate an AEM project and understand how its parts fit together.

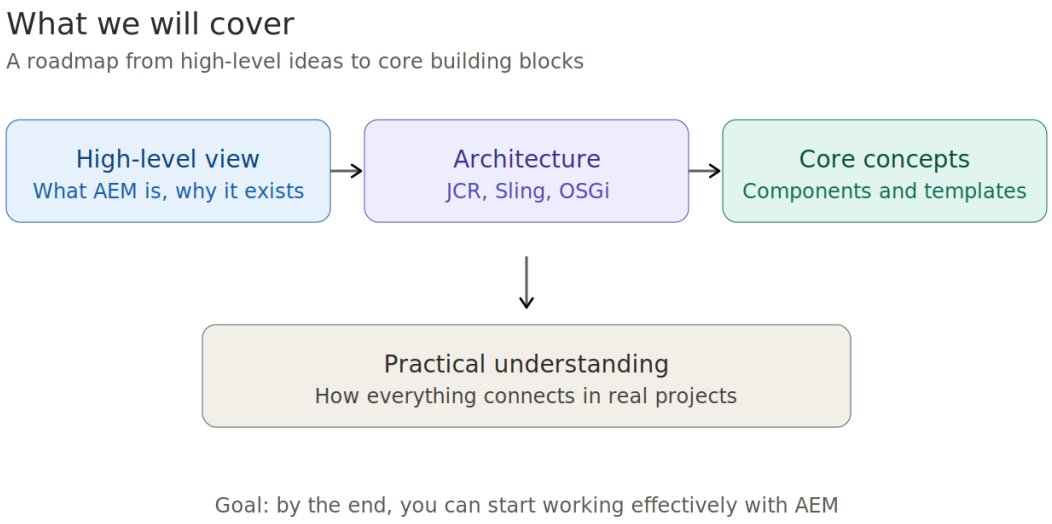

Figure 1. Roadmap of what this guide covers.

2. What is AEM?

AEM is an enterprise content management system (CMS), which means it is a platform used to manage digital content and deliver it across multiple channels. In simple terms, it helps companies build and manage digital experiences at scale.

The four main pillars

- Web content management — pages, sites, and multi-channel delivery.
- Digital asset management — images, videos, and documents.
- Forms — customer-facing data capture.
- Marketing tools — personalization and campaign management.

AEM is part of Adobe Experience Cloud, so it integrates with tools like Adobe Analytics and Adobe Target. It is widely used in industries such as banking, retail, healthcare, and government.

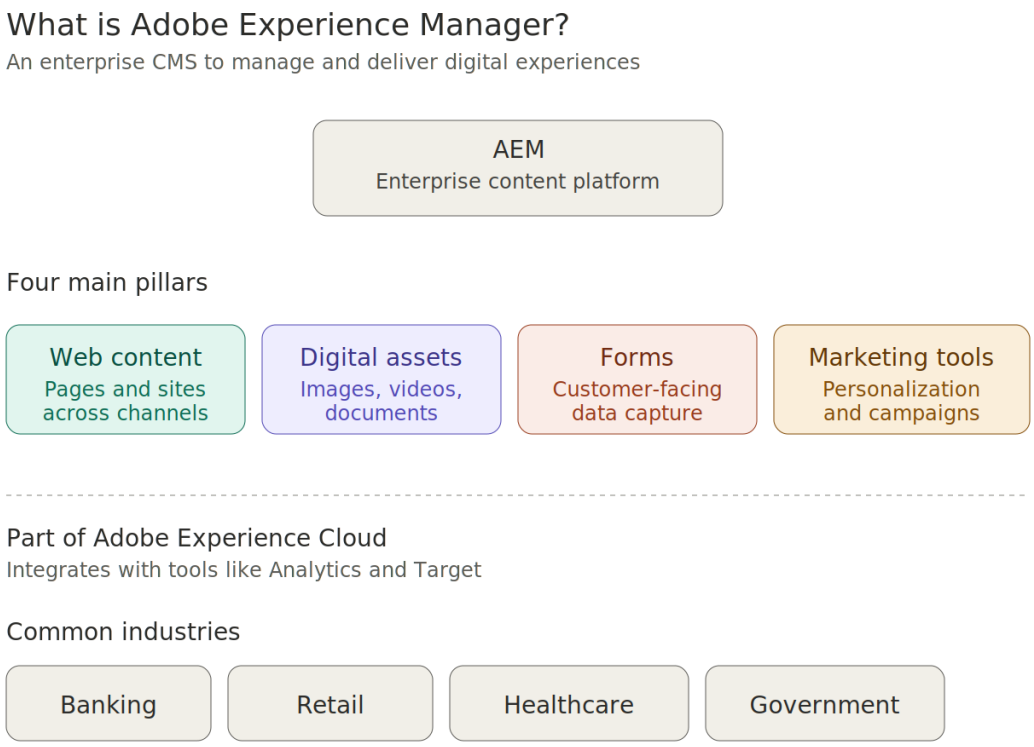

Figure 2. AEM at a glance: pillars and ecosystem.

3. AEM architecture

AEM is built on three main layers. Think of it as a stack:

- JCR (Java Content Repository) — where content is stored.
- Apache Sling — resolves incoming URLs to content.
- OSGi — hosts services and business logic.

Basic request flow

When a user opens a page, the request travels through these layers in a predictable sequence:

- An HTTP request comes in.
- Sling interprets the URL and identifies the target.
- The resource is located in the repository (JCR).
- The component responsible for rendering is identified.
- OSGi executes the business logic.
- A response is generated and returned to the user.

If you understand this flow, you understand a big part of AEM.

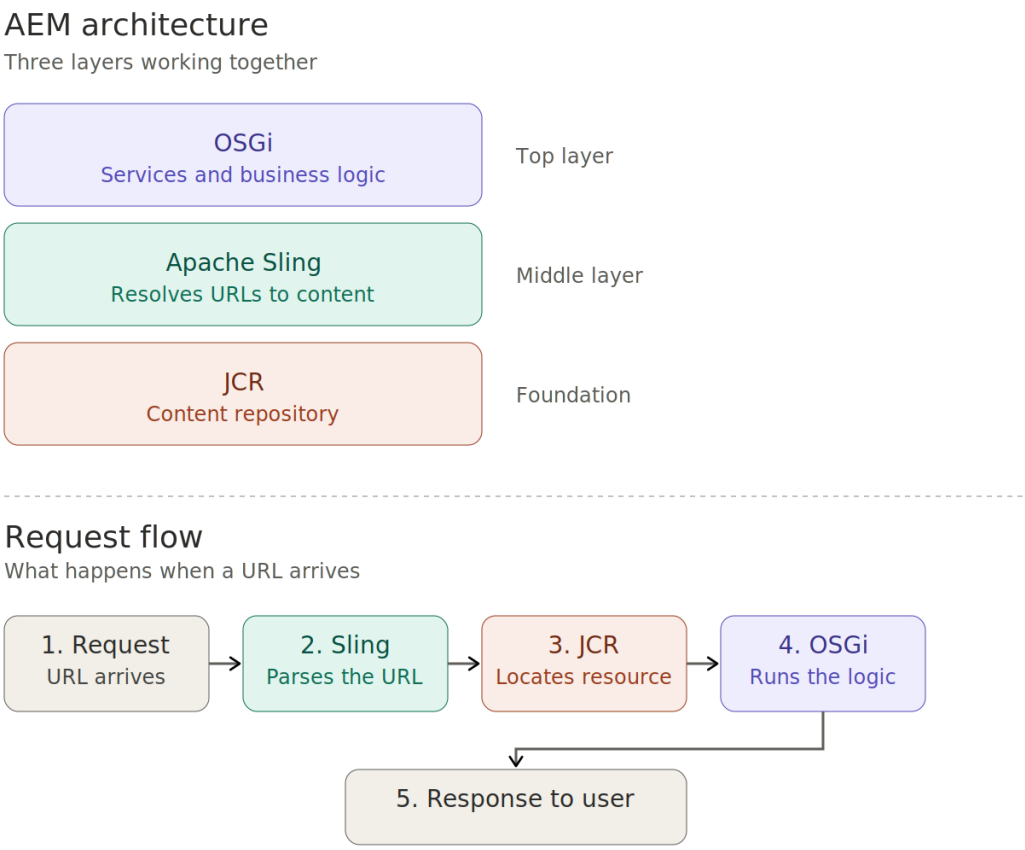

Figure 3. The three-layer stack and the request flow.

4. JCR (Content Repository)

Content in AEM is stored in a tree structure, similar to a file system. Everything in AEM revolves around content, so this is a fundamental concept.

Key paths

- /content — website pages.
- /apps — your custom code.
- /libs — AEM internal code (never modify).
- /conf — configurations.
- /var — runtime data.
- /home — users and groups.

Each page has a special node called jcr:content where its properties live: title, template, layout, and the components that make up the page.

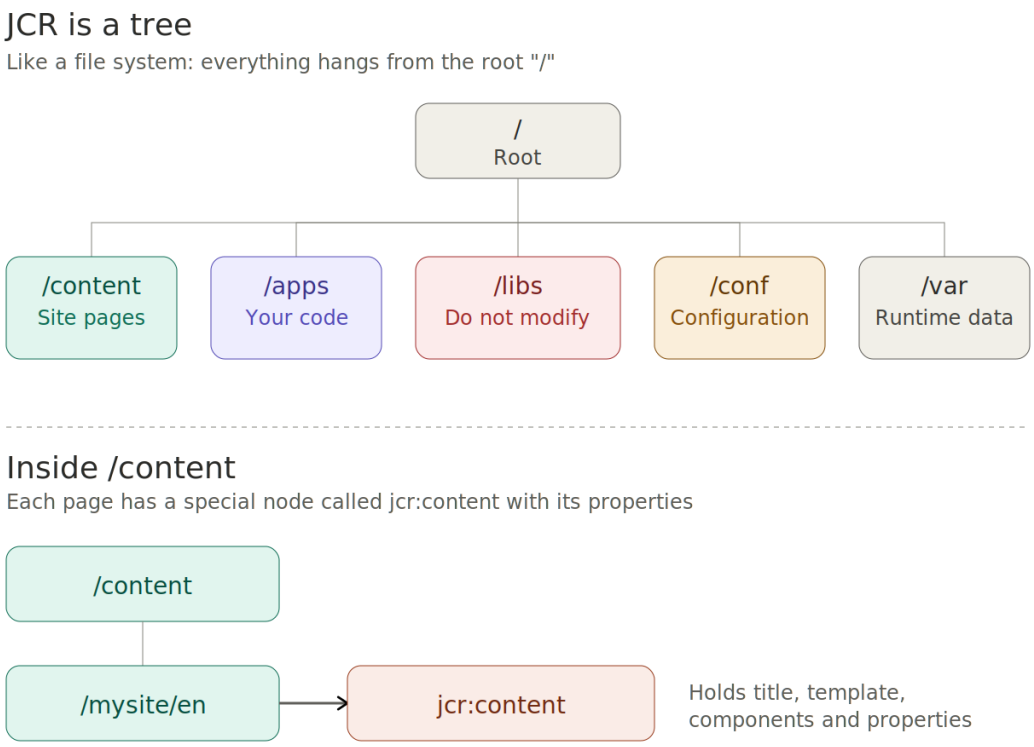

Figure 4. JCR tree: top-level paths and the jcr:content node.

5. Apache Sling

Sling is the engine that connects URLs to content. It follows a RESTful approach and uses a key property called sling:resourceType, which tells AEM which component should render a resource.

Anatomy of a Sling URL

Take a URL like /content/mysite/en.mobile.html. Sling reads it in three parts:

- Path — /content/mysite/en — where the content lives in the repository.
- Selector — mobile — variant or rendering mode.
- Extension — html — output format (could also be json, xml, and so on).

This structure gives AEM a lot of flexibility in how the same content can be delivered.

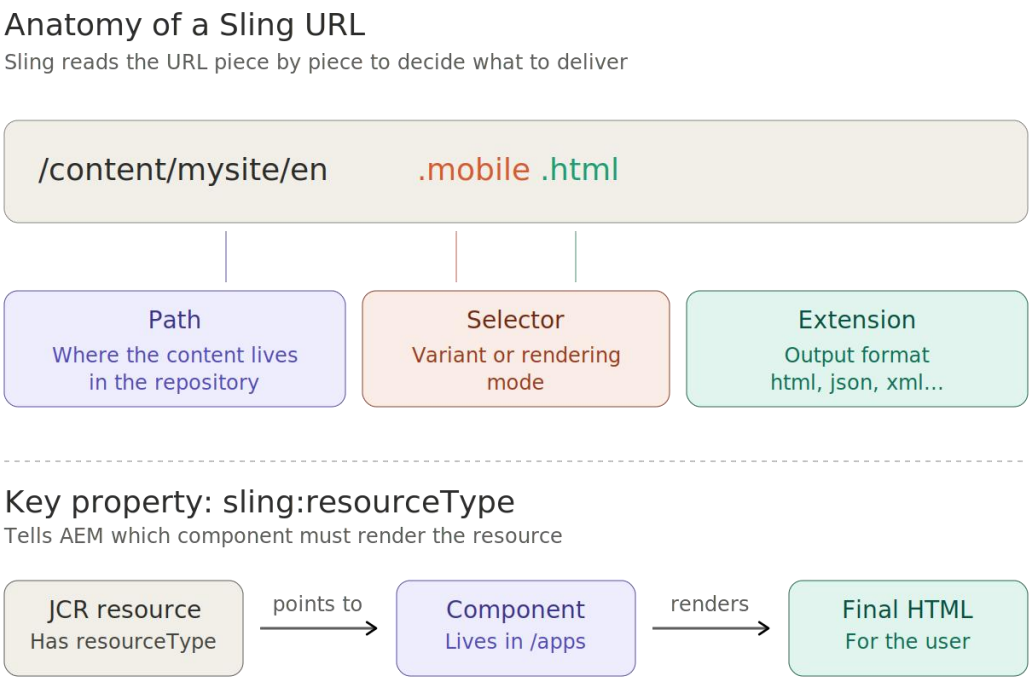

Figure 5. How Sling decomposes a URL and links it to a component.

6. OSGi

OSGi is the container where business logic lives. It manages services and configurations, organized in modules called bundles.

Why OSGi matters

- Hot deploy — bundles can be installed, updated, or removed without restarting the server.
- Dependency injection — services discover and consume each other cleanly.
- Per-environment configuration — settings can vary by dev, stage, and production.

This combination is what makes AEM highly dynamic and modular at runtime.

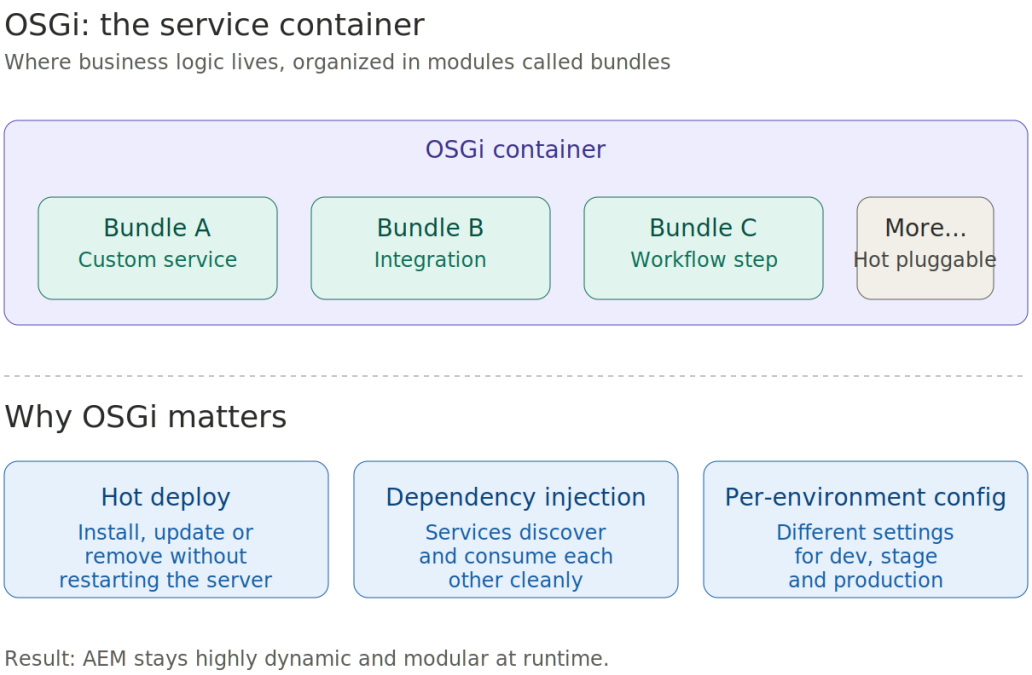

Figure 6. OSGi as a container of bundles, with its key benefits.

7. Components

Components are the basic building blocks in AEM. You can think of them as LEGO pieces that authors combine to build pages.

Anatomy of a component

- Definition — resourceType, name, and metadata.
- HTL template — HTML markup that produces the rendered output.
- Author dialog — the form authors use to fill in content.
- CSS / JS — styles and client-side behavior.

HTL: HTML Template Language

HTL is the modern templating standard in AEM. It is secure by default, protects against cross-site scripting (XSS), and replaces older technologies like JSP.

There are also Core Components, a library of ready-to-use, production-quality components maintained by Adobe.

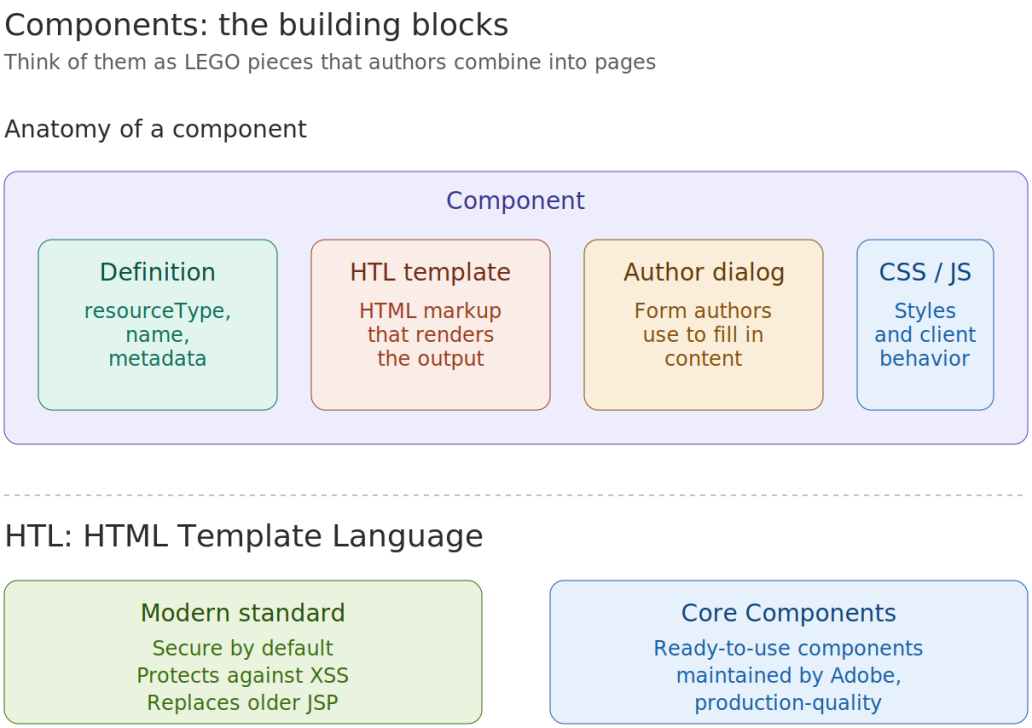

Figure 7. Component anatomy and the role of HTL and Core Components.

8. Editable templates

Templates define the structure that pages start with. They separate developer concerns from authoring concerns.

Two main areas

- Structure — locked areas like header and footer that authors cannot edit.
- Initial content — default editable content area where authors freely add and arrange components.

Policies define the rules

- Allowed components — which components authors can drop in a given area.
- Available styles — predefined visual variations authors can apply.

This creates a clear separation between developers and content authors, while keeping authors within safe boundaries.

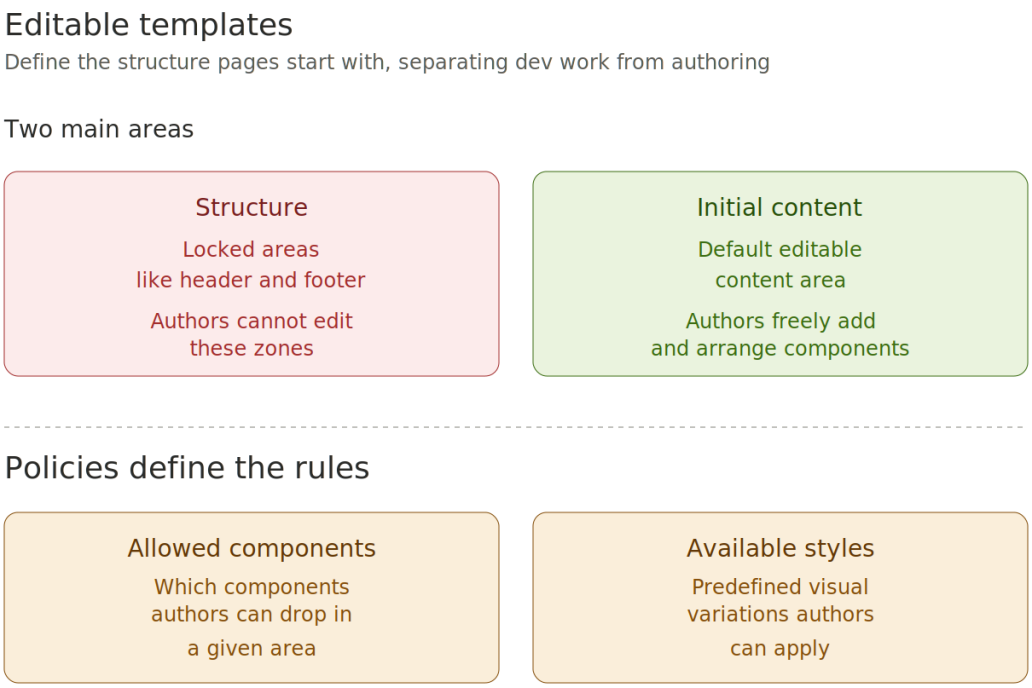

Figure 8. Editable templates: structure, initial content, and policies.

9. Content Fragments vs Experience Fragments

It is important to understand the difference between these two reusable building blocks.

Content Fragment

- Focused on data.
- No design or layout.
- Ideal for APIs or headless delivery.

Experience Fragment

- Includes layout and design.
- Reusable visual section.
- Ideal for banners or page sections.

In short: a Content Fragment is the message, while an Experience Fragment is the message together with how it looks.

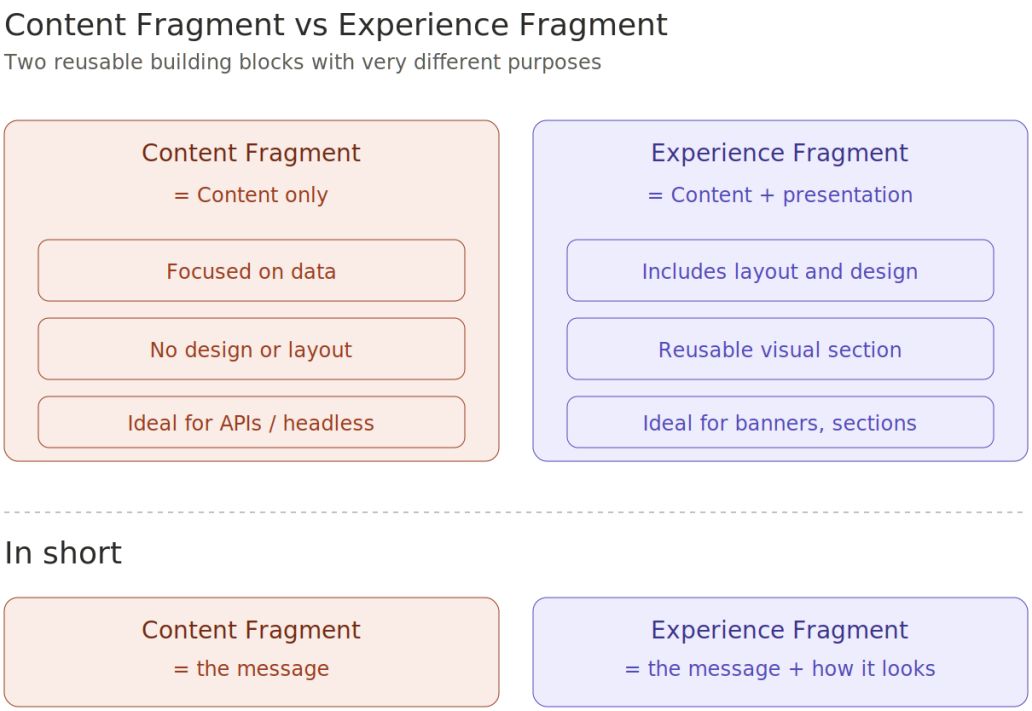

Figure 9. Side-by-side comparison of Content and Experience Fragments.

10. Sling Models

Sling Models are Java classes used to retrieve data from the repository in a clean, declarative way. They rely on annotations for dependency injection.

Key annotations

- @ValueMapValue — injects properties from the resource.
- @ChildResource — injects child nodes.
- @OSGiService — injects an OSGi service.
- @PostConstruct — runs initialization logic after injection.

Sling Models replace logic that would otherwise live in views. The result is cleaner, more maintainable code with a clear separation between data preparation and rendering.

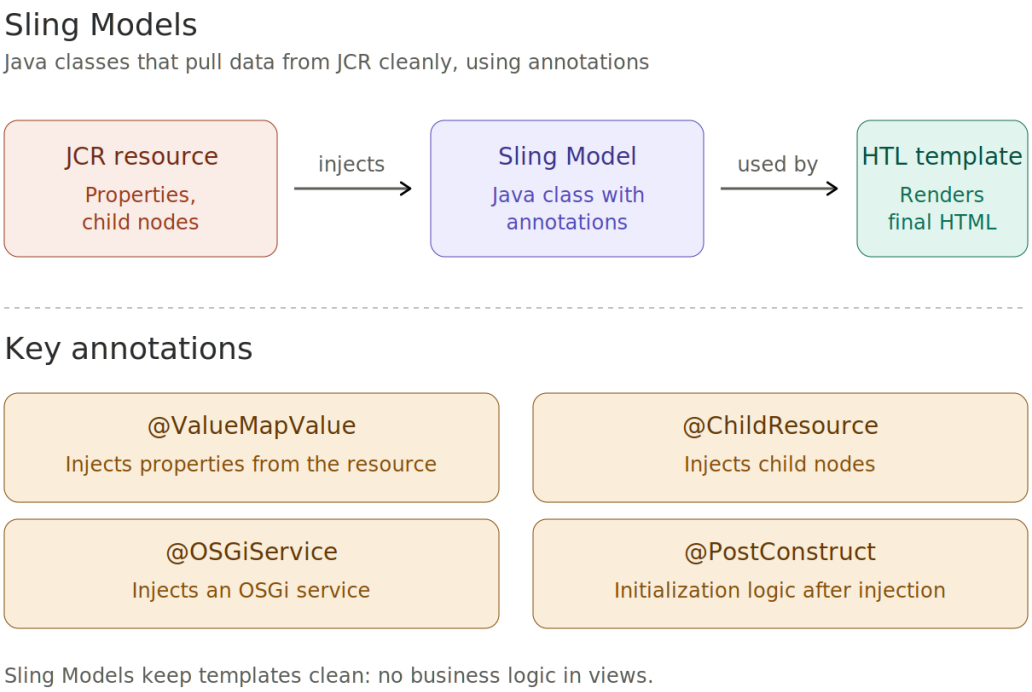

Figure 10. Sling Models bridge the JCR resource and the HTL template.

11. AEM as a Cloud Service

AEM can run as a cloud service, which introduces some important changes compared with the classic on-premise model.

What changes

- Infrastructure is managed by Adobe — no servers to install or maintain yourself.
- Automatic updates — always running on a current version of AEM.
- Auto-scaling — capacity adjusts to traffic automatically.

Important rules

- /apps and /libs are read-only at runtime — all changes go through deployments.
- Configuration uses environment variables instead of per-instance configs.
- RDE (Rapid Development Environment) is available for fast iteration without full deployments.

This model enforces modern best practices and pushes teams toward clean, repeatable deployments.

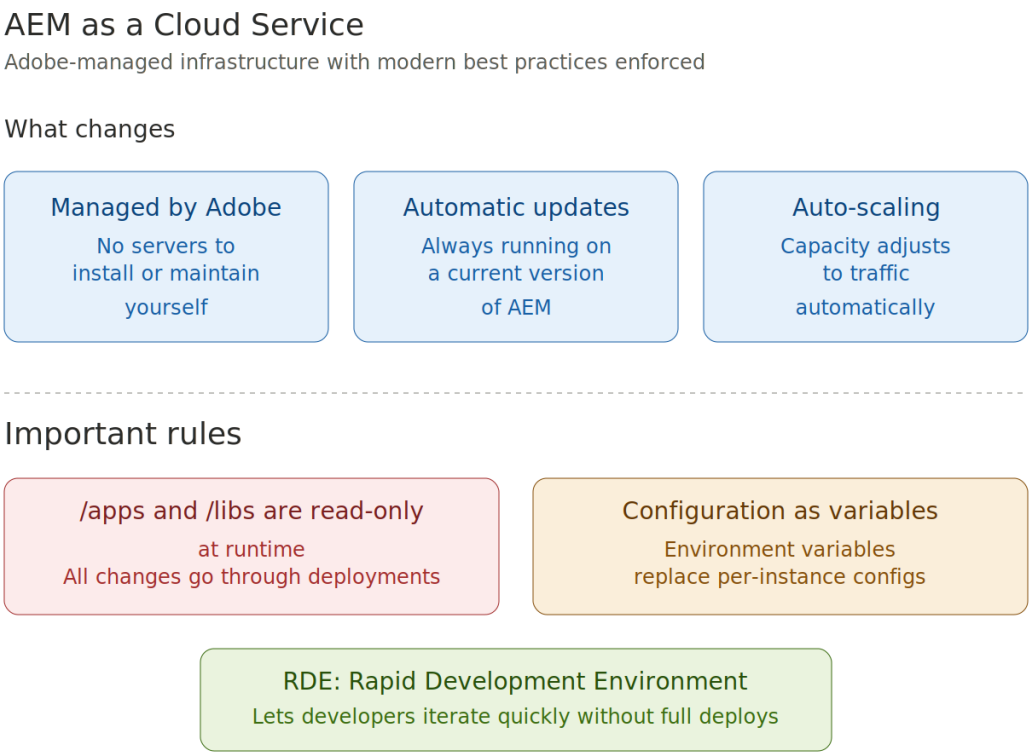

Figure 11. AEM as a Cloud Service: changes and rules to keep in mind.

12. Author vs Publish

AEM has two main runtime environments that separate content creation from content delivery.

Author

- Where content is created and managed.
- For internal use only — authors and editors.

Publish

- Where content is delivered to end users.
- Public-facing — what visitors actually hit.

Content moves from Author to Publish through a publishing process called replication.

The Dispatcher

In front of Publish there is the Dispatcher, which handles caching, improves performance, and adds a security layer. It is the first thing visitors hit, protecting Publish from heavy load.

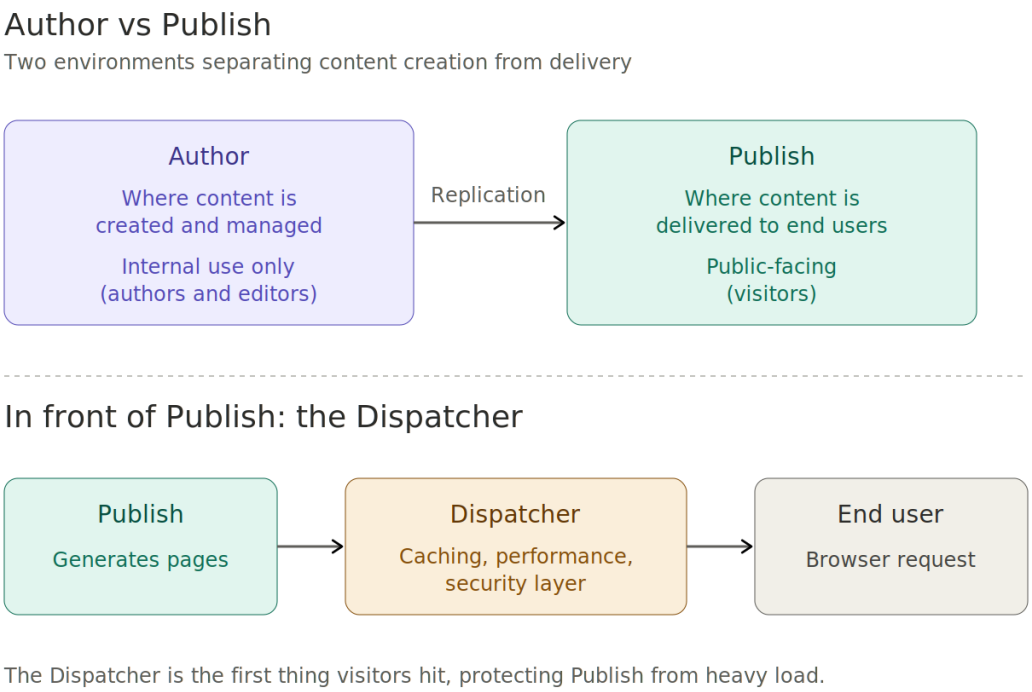

Figure 12. Author, Publish, and the Dispatcher in front of Publish.

13. Closing

AEM is a powerful platform built on three foundations:

- Structured content — the JCR tree.
- Dynamic request resolution — Apache Sling.
- Modular services — OSGi.

The key takeaway is understanding how everything connects:

- How content links to components, via sling:resourceType.
- How requests are processed, through Sling, JCR, and OSGi.
- How projects are structured, separating /apps, /content, and /conf.

If you understand these fundamentals, you can start working effectively with AEM and dig deeper into each topic with confidence.

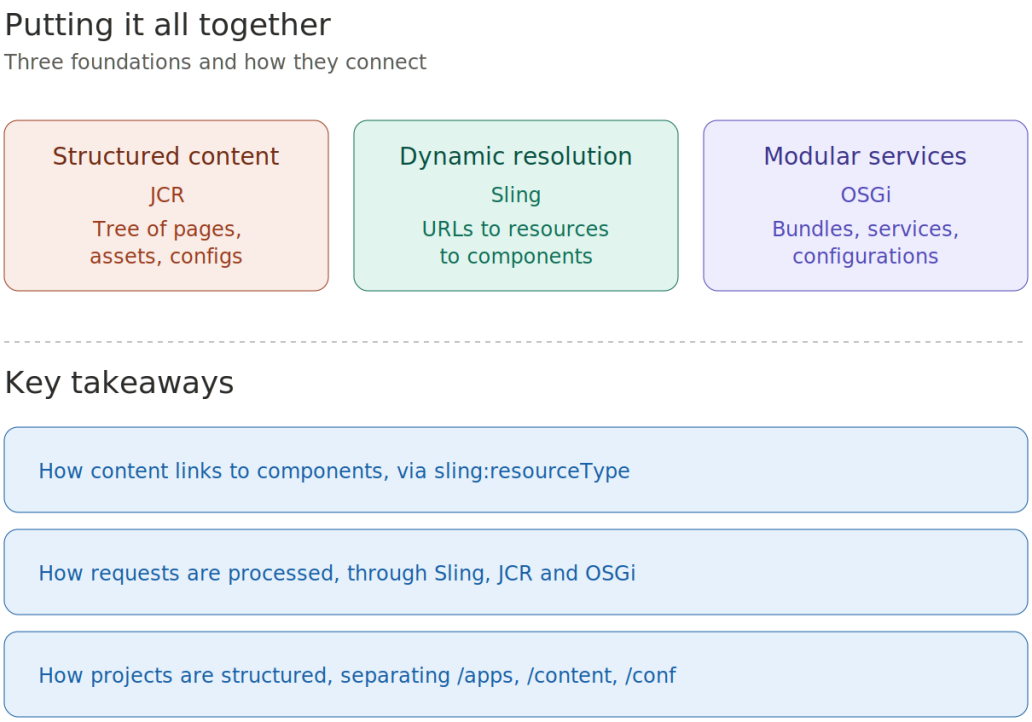

Figure 13. The three foundations and the key takeaways.